Avalon-MM SDRAM Controller IP Core

User Guide

IP core	`altera_avalon_mm_sdram_controller`
Core version	1.0
Document version	1.0
Date	August 2026
Target	Intel FPGA / Quartus Prime, Platform Designer

SDR SDRAM controller for Avalon-MM, with one open row per bank. Source-available at github.com/monkstein88/altera-ip-cores.

Contents

1. About this core

1.1 Features

- **One open row per bank.** Four rows held open at once rather than one, so an access that changes bank costs no row command.
- **Read/write turnaround without a row cycle.** 0 cycles write→read, CAS+1 read→write, as the device datasheet describes.
- **Look-ahead row activation.** The row for the *next* buffered access is opened or closed while the current one is still being served.
- **Postponed refresh.** Up to `REF_MAX_PEND` refreshes are deferred to a quiet moment, then forced through.
- **Parameterised in time, not cycles.** Device timings are picoseconds; cycles are derived inside the HDL.
- **Drop-in.** Same `s1` slave, same `wire` conduit, same default address map as the core it replaces.

1.2 What it is for

Mixed and scattered traffic — which is what a CPU generates. Sequential streaming was already at 97% of the bus limit before this core existed and is not where the headroom was. See section 7.

1.3 Status

Verified in simulation, synthesised, closing timing, and run on two boards. The core compiles through Quartus 18.1.1 Standard for the DE10-Lite's MAX 10 and the DE0-Nano's Cyclone IV E, meets a 100 MHz constraint on both, and has been programmed into both: eight of eight RTL scenarios and ten of ten Nios II checks on each. The two parts differ in column width and capacity — 9 bits and 32 MByte against 10 bits and 64 — so the address decode and the preset mechanism are exercised, not just one geometry. Resource and f_MAX figures are in section 7.4 and are reproducible.

2. Getting started

2.1 Adding the component

Add the repository to the IP search path — **Tools ▸ Options ▸ IP Search Path**, or `--search-path` on the command line — and the component appears in the IP Catalog under **Memory Interfaces and Controllers / Custom** as *Avalon-MM SDRAM Controller (per-bank rows)*.

2.2 Start from a preset

Select the preset for your part rather than typing timings in:

```
add_instance sdram altera_avalon_mm_sdram_controller
apply_preset sdram "ISSI IS42S16320D-7 - DE10-Lite 64 MByte"
```

A preset carries the geometry, the timings and the refresh figures for one device, taken from its datasheet once. Three are supplied:

Preset name	Part	Size	Banks x rows x cols x bits	On a board here
`ISSI IS42S16320D-7 - DE10-Lite 64 MByte`	IS42S16320D-7	64 MB	4 x 8192 x 1024 x 16	yes
`ISSI IS42S16160B-7 - DE0-Nano 32 MByte`	IS42S16160B-7	32 MB	4 x 8192 x 512 x 16	yes
`Micron MT48LC4M16A2-75 - 8 MByte simulated only`	MT48LC4M16A2-75	8 MB	4 x **4096** x 256 x 16	no

The third is on neither board. It is supplied because it is the only one of the three with a different **row count**, and therefore a different refresh interval: 4,096 rows in 64 ms is a tREFI of 15.625 us against the ISSI parts' 7.8125. Two presets that happen to share a row count cannot show whether the refresh arithmetic follows the part or a default - and when this one was added it turned out that in two places it did not.

Only parts whose timing has been checked against a datasheet *and* exercised through `benchmark/` and the testbench are supplied; adding one is a block of XML in `altera_avalon_mm_sdram_controller.qprs`. Because a preset is a datasheet transcribed, both are held in place from two directions: `doc/tools/check_facts.py` compares each preset against the copy the benchmark and testbench carry, and the regression simulates each part's geometry and timings rather than only linting them.

> **Caution**
> There is deliberately no "generic SDRAM" preset. A timing constant that is one speed grade optimistic produces a design that passes every simulation, works on the bench, and corrupts data once the board is warm.

2.3 Connecting it

Interface	Connect to
`clk`, `reset`	Your clock source. **The clock rate is read from it**, so no frequency is typed in anywhere
`s1`	The master, or the interconnect
`wire`	Export to the top level, and constrain the SDRAM pins

2.4 What is checked at generation time

The component refuses, or warns about, four configurations that would otherwise build and be wrong on hardware:

Condition	Result
`SA_BITS` too small for `ROW_BITS`, or below 11	Error
tRAS longer than tRC, or tRRD longer than tRC	Error
A refresh interval the controller cannot sustain	Error
`ADDR_MAP` set to conventional	Warning — every address moves

It also reports the memory size and the exact cycle counts the HDL will derive at your clock, so they can be read against the datasheet:

```
Info: Memory: 64 MByte - 4 banks x 8192 rows x 1024 columns x 16 bits.
Info: At 143.000 MHz the HDL will use: tRC=9 tRAS=6 tRP=3 tRCD=3 tRRD=3
      tWR=3 tRFC=9 cycles, CAS=3, one refresh every 1118 cycles.
```

3. Functional description

3.1 Initialisation

After reset the controller holds `za_waitrequest` high and runs the JEDEC power-up sequence: wait `T_INIT_US`, PRECHARGE ALL, `INIT_REFS` AUTO REFRESH commands, then LOAD MODE REGISTER. Only then is a transfer accepted.

The mode register is written with burst length 1, sequential burst type, and the configured CAS latency.

3.2 Serving a transfer

One command is issued per cycle, chosen in priority order — see Figure 3 in the block-diagram document. In summary: serve the head if its row is open, close the row if it is the wrong one, open the bank if it is closed, otherwise prepare the next access.

3.3 Read/write turnaround

A READ issued at cycle *T* has the device driving DQ at *T+CAS*. A WRITE drives DQ in its own cycle, so the earliest safe write is *T+CAS+1*. The other direction is free — the datasheet permits write data to be immediately followed by a READ command.

This is why mixed traffic settles at about 79 MB/s rather than 194: a write/read pair costs five cycles at CAS 3, and two accesses per five cycles is 80 MB/s. That is the device, not the scheduler.

3.4 Refresh

A timer accumulates outstanding refreshes at the interval implied by `REF_ROWS` and `REF_PERIOD_MS`. They are issued when the command buffer is empty, or forced once `REF_MAX_PEND` have accumulated. A refresh precharges every bank, so all open rows are lost and reopened as needed.

3.5 Byte enables

`az_be_n` is active low and is driven onto `zs_dqm` for writes. DQM is held low at all other times, which satisfies the SDR read-output enable requirement.

4. Parameters

4.1 Geometry

Parameter	Type	Default	Range	Description
`DATA_BITS`	Integer	16	8, 16, 32	SDRAM data bus width. One DQM bit per byte lane
`ROW_BITS`	Integer	13	10:15	Row address width
`COL_BITS`	Integer	10	8:11	Column address width. Column bit 10 steps over A10
`BANK_BITS`	Integer	2	1, 2, 3	Bank address width. Also how many rows can be open at once
`SA_BITS`	Integer	13	11:15	Address pins on the device. At least `ROW_BITS`, and at least 11
`ADDR_W`	Integer	25	derived	Avalon word address width — row + column + bank. Do not override

4.2 Device timing

Given in **picoseconds** in the HDL and in **nanoseconds** in Platform Designer, which scales them. See section 4.5 for why.

Parameter	Default	Description
`T_RC_PS`	60 000	ACTIVATE to ACTIVATE, same bank
`T_RAS_PS`	37 000	ACTIVATE to PRECHARGE, same bank
`T_RP_PS`	15 000	PRECHARGE to ACTIVATE, same bank
`T_RCD_PS`	15 000	ACTIVATE to READ or WRITE
`T_RRD_PS`	14 000	ACTIVATE to ACTIVATE, different bank
`T_WR_PS`	14 000	Write recovery: last write data to PRECHARGE
`T_MRD_PS`	14 000	LOAD MODE REGISTER to next command
`T_RFC_PS`	60 000	AUTO REFRESH to next ACTIVATE or REFRESH
`CAS_LAT`	3	CAS latency, 2 or 3

4.3 Initialisation and refresh

Parameter	Default	Range	Description
`T_INIT_US`	100	1:10000	Power-up wait before the first command, microseconds
`INIT_REFS`	8	2:15	AUTO REFRESH commands during initialisation
`REF_ROWS`	8192	512:32768	Rows the device refreshes per period
`REF_PERIOD_MS`	64	1:1000	Refresh period, milliseconds
`REF_MAX_PEND`	8	1:8	Refreshes that may be postponed. 1 refreshes immediately

4.4 Controller options

Parameter	Default	Range	Description
`CLK_KHZ`	100 000	derived	Taken from the connected clock source. Every timing is converted against it
`ADDR_MAP`	0	0, 1	0 = compatible with the Intel core, 1 = conventional `{row, bank, column}`
`FIFO_DEPTH`	8	2, 4, 8, 16, 32	Buffered commands. Also what look-ahead looks into
`LOOKAHEAD`	1	0, 1	Open or close the next access's row early
`RD_EXTRA_LAT`	0	0:3	Extra read-capture delay. Zero is correct for a direct connection. Also lengthens the read-to-write turnaround by the same amount
`WR_TURNAROUND_EXTRA`	0	0:3	Dead cycles added between a READ and the next WRITE, beyond the datasheet minimum of CAS+1. See below

The read-to-write turnaround is `CAS_LAT + RD_EXTRA_LAT + 1 + WR_TURNAROUND_EXTRA` cycles, and the default of `CAS_LAT + 1` is the datasheet **minimum**, not a comfortable figure. Both supplied parts permit a WRITE on the clock edge immediately following the last read data element only *"provided that I/O contention can be avoided"*, and warn that the device driving the input data may go Low-Z before the SDRAM DQs go High-Z — `tLZ` is 0 ns minimum and `tHZ` is up to 5.4 ns at the -7 grade.

The datasheets offer two remedies. One is DQM, asserted two to three clocks ahead to force the device's outputs to High-Z; that one is **unavailable here**, because the mode register programs burst length 1, so the only data element in flight is the one being read and blanking it destroys the transfer. The other is a cycle of dead time, which is what `WR_TURNAROUND_EXTRA` buys.

Zero is what both supplied boards run and what every published measurement was taken at. Each added cycle costs roughly 15% of alternating read/write throughput and nothing at all on same-direction streaming, which is already at 97–99% of the bus. Raise it if your board's DQ flight time makes that overlap real, and measure the result on `benchmark/` rather than assuming.

4.5 Why picoseconds, and why not cycles

Cycle counts are deliberately not exposed. A timing parameter rounded the wrong way is silent data corruption at temperature months later, not a clean failure, and pushing that arithmetic onto every integrator guarantees someone gets it wrong.

Picoseconds rather than a `real` number of nanoseconds because **Platform Designer emits a FLOAT parameter into the generated Verilog as a quoted string** — `.T_RC_NS("60.0")`. Assigned to a `parameter real`, that string is its ASCII bytes read as a number: 60.0 arrives as 909127216.0, and a 6-cycle tRC becomes 90 million. Integers cross the tool boundary intact.

> **Caution**
> If you add a parameter to this core, check the generated wrapper and confirm it arrives unquoted.

5. Signals

20 ports.

5.1 Clock and reset

Signal	Direction	Width	Description
`clk`	Input	1	All logic is in this domain
`reset_n`	Input	1	Asynchronous assert, synchronous deassert

5.2 `s1` — Avalon-MM slave

Word-addressed, `isMemoryDevice`, variable read latency with `readdatavalid`.

Signal	Direction	Width	Avalon role
`az_addr`	Input	`ADDR_W`	`address` (words)
`az_be_n`	Input	`DATA_BITS/8`	`byteenable_n`
`az_cs`	Input	1	`chipselect`
`az_data`	Input	`DATA_BITS`	`writedata`
`az_rd_n`	Input	1	`read_n`
`az_wr_n`	Input	1	`write_n`
`za_data`	Output	`DATA_BITS`	`readdata`
`za_valid`	Output	1	`readdatavalid`
`za_waitrequest`	Output	1	`waitrequest`

5.3 `wire` — SDRAM conduit

Signal	Direction	Width
`zs_addr`	Output	`SA_BITS`
`zs_ba`	Output	`BANK_BITS`
`zs_cas_n`	Output	1
`zs_cke`	Output	1
`zs_cs_n`	Output	1
`zs_dq`	Bidir	`DATA_BITS`
`zs_dqm`	Output	`DATA_BITS/8`
`zs_ras_n`	Output	1
`zs_we_n`	Output	1

6. Address map

See Figure 4. At the default geometry, `ADDR_MAP 0` gives `bank = {addr[24], addr[10]}`, `row = addr[23:11]`, `column = addr[9:0]`.

The Avalon address is a **word** address, matching the core this replaces, so a system's existing address assignments carry over unchanged.

7. Performance

Measured on `benchmark/`, identical stimulus, both controllers. ISSI IS42S16320D at 100 MHz, 16-bit bus, CAS 3, 4096 operations per pattern. Theoretical peak 200 MB/s.

Pattern	Intel's core	This core	Speedup
seq write	194.3	198.3	1.02×
seq read	194.0	196.1	1.01×
seq read/write	21.9	78.9	3.6×
same-row rd/wr	21.9	78.9	3.6×
bank+row walk	21.9	65.5	3.0×
4-bank same row	21.9	**194.7**	**8.9×**
random	22.0	44.5	2.0×

0 data errors, 0 timing violations, both controllers.

Sequential streaming was already finished and is essentially unchanged. The gain is entirely in mixed and scattered traffic.

> **Note**
> These are simulation figures. The first two rows reproduce the 194 MB/s the SDRAM example measured on hardware, which is the check that the harness measures the right thing — but no figure in this table has itself been observed on a board.

7.4 Resources and f_MAX

Quartus 18.1.1 Standard, MAX 10 `10M50DAF484C7G`, Slow 1200 mV 85 °C model. Both controllers fitted standalone under the same constraints. Every figure is the median of five fitter seeds, with the range beside it, because placement moves f_MAX here by more than most RTL changes do. Reproduce the table with `doc/tools/measure_fit.sh`.

	Intel's core	This core
Logic elements	351 (348–352)	1,345 (1,337–1,366)
Registers	284	787
f_MAX	113.5 MHz (109.5–123.4)	102.1 MHz (99.7–106.0)

The complete DE10-Lite demonstration — this controller plus sequencer, master, PLL, seven-segment displays and JTAG probes — occupies 3,306 logic elements and 1,875 registers, 7% of the device, and closes its 100 MHz constraint with 0.601 ns of setup slack. The SDRAM interface paths close with 2.035 ns. The DE0-Nano demonstration, on a Cyclone IV E, occupies 3,325 logic elements and closes with 1.357 ns.

Intel's core is generated output that this repository does not track, so its column describes whatever its own DE10-Lite example last generated.

> **Caution**
> the throughput table in 7.3 assumes 100 MHz for both controllers. Cycle counts are a property of the scheduler; megabytes per second are not. This core reaches 102.1 MHz and the core it replaces reaches 113.5, so the ratios hold at 100 MHz and below and stop holding above it. A system already running Intel's controller above 102.1 MHz cannot substitute this one without lowering its clock.

The critical path is the loop from the registered command-buffer head, through the scheduler's priority chain, through the pop decision, and back into that register. Buffering the FIFO output is what made 100 MHz reachable at all: with the head taken combinationally from the array, the multiplexer and the entire priority chain shared one cycle, and f_MAX was 83 MHz.

8. Verification

Flow	What it covers	Status
`simulation/verilator/run_sim.sh`	Lint of RTL, checker and model; timing-checker self-test; testbench across 14 configurations including three clock rates and all three supplied parts; lint in 4 geometries; Quartus Analysis & Synthesis	**23 checks, passing**
`tb/avalon_mm_sdram_controller_tb.sv`	168 checks per configuration, asserting on the command stream as well as the data	Passing
`tb/avalon_mm_sdram_controller_sva.sv`	Avalon protocol, command legality, DQ contention, row bookkeeping	Passing
`tb/sdram_timing_check.sv`	tRC, tRAS, tRP, tRCD, tRRD, tWR, tMRD, tRFC, read-to-write turnaround, refresh interval	Passing, with a 23-check threshold self-test
`benchmark/`	Throughput against the core being replaced	Passing
`example/de10_lite_rtl`	Board-level demonstration, 9 phases	61 checks passing in simulation
`simulation/questa/run_sim.tcl`	Coverage and assertion non-vacuity, same 15 configurations	23 assertion instances, **none vacuous**, 100% FSM state and transition
Quartus	Synthesis, fit, timing closure, bitstream	100 MHz met with 1.011 ns slack (DE0-Nano), 0.208 ns (DE10-Lite)
Hardware	Retention, refresh and full-device marches on silicon	**Both boards: 8/8 RTL and 10/10 Nios II each**

8.1 The testbench checks the mechanism, not only the data

A controller that closed and reopened a row before every access would return correct data and pass any integrity check. So the testbench counts commands: four banks with one row open each, accessed in rotation, is asserted to cost exactly four ACTIVATEs and zero PRECHARGEs.

8.2 Proven by fault injection

Six faults were introduced into the controller and each was caught by the layer meant to catch it:

Fault	Caught by
Byte enables ignored	Directed test
Read/write turnaround removed	SVA
tRCD gate removed	Timing checker
ACTIVATE forgets its row	SVA
Refresh never forced under load	SVA
Read data captured a cycle late	Directed test

9. Limitations and known gaps

Limitation	Detail
No bursting	The slave is non-bursting; every transfer is one word. Bursts would amortise the read/write turnaround, and would also end the drop-in guarantee - see the decision recorded in the README
No reordering	Transfers are served in order. A write buffer grouping same-direction accesses would collect the same 1.7-2x on alternating traffic without changing the interface; it was costed and deliberately not built, because no workload measured on either board reaches the turnaround
One chip select	Multi-device configurations are not supported
Single clock domain	No clock-crossing on the slave port
Two device presets	The DE10-Lite's IS42S16320D-7 and the DE0-Nano's IS42S16160B-7. Others need datasheet figures
f_MAX headroom is thin	104.8 MHz standalone against a 100 MHz target. The critical path now starts at the tRC counter and runs through `act_ok_v` and the priority chain into the row bookkeeping; the counters could be given registered reaches-zero flags the way the row match already has one, and beyond that shortening the chain means splitting it across two cycles

10. Revision history

Document version	Core version	Date	Change
1.0	1.0	August 2026	First release